Sprawozdanie KM1: definicja problemu, dobór danych i projekt eksperymentu

Oct 28, 2026 · @Bartek

1. Wprowadzenie i cel projektu

Rozpoznawanie znaków drogowych to zadanie, z którym radzi sobie dziś praktycznie każdy nowszy samochód z systemami wspomagania kierowcy. Kamera w aucie rzadko dostaje jednak ładne, ostre zdjęcie w pełnym słońcu. Obraz bywa rozmyty przez ruch, zaszumiony o zmierzchu albo po prostu ciemny, a sam znak potrafi być częściowo zasłonięty przez gałąź czy naklejkę.

Na zbiorze GTSRB, z którego korzystam, dobre sieci konwolucyjne dochodzą do około 99% skuteczności. Kolejna próba wyciśnięcia z niego ułamka procenta nie wydała mi się więc szczególnie ciekawa. Zamiast tego postanowiłem sprawdzić, co dzieje się z tą skutecznością, kiedy warunki przestają być idealne, i skupić się na odporności modeli.

Pytanie badawcze: czy uczenie transferowe i odpowiednio dobrana augmentacja danych ograniczają spadek skuteczności klasyfikatora znaków drogowych na zakłóconych zdjęciach?

Żeby odpowiedzieć na nie liczbami, a nie wrażeniem, wytrenuję kilka wariantów sieci i porównam ich wyniki na czystych zdjęciach oraz na zdjęciach, które celowo i w kontrolowany sposób zepsuję. Z tego porównania wyjdzie, które warianty tracą najmniej i co je od pozostałych odróżnia.

W tym etapie chodziło mi przede wszystkim o to, żeby jasno ustalić, co dokładnie chcę sprawdzić i w jaki sposób udowodnię, że rozwiązanie działa. Wszystko, co zrobię w KM2 i KM3, będzie wynikać z ustaleń opisanych poniżej.

2. Zbiór danych

Wybrałem zbiór **GTSRB** (German Traffic Sign Recognition Benchmark), przygotowany przez zespół z Ruhr-Universität Bochum na konkurs IJCNN 2011 [1]. Zdjęcia pochodzą z nagrań kamery samochodowej z niemieckich dróg, a każde z nich to pojedynczy znak wycięty z klatki filmu razem z niewielkim marginesem tła. Dane pobrałem z serwisu Kaggle [2], gdzie są dostępne jako pliki PNG z opisem w plikach CSV.

Element	Zawartość
`Train/`	39 209 zdjęć treningowych w 43 podfolderach (jeden na klasę)
`Test/`	12 630 zdjęć testowych
`Train.csv`, `Test.csv`	rozmiar zdjęcia, współrzędne ramki ze znakiem (ROI), numer klasy, ścieżka do pliku
`Meta/`	wzorcowe piktogramy wszystkich 43 klas

Dlaczego właśnie ten zbiór

Przy wyborze zależało mi na kilku rzeczach:

- **Sprawdzone źródło.** GTSRB to uznany benchmark opisany w recenzowanej publikacji, z ręcznie weryfikowanymi etykietami, więc nie muszę się zastanawiać, czy dane są wiarygodne.
- **Rozsądna wielkość.** Ponad 51 tysięcy zdjęć to wystarczająco dużo, żeby porównanie architektur miało sens, a jednocześnie na tyle mało, że trening na moim laptopie trwa około godziny, a nie całe noce.
- **Oficjalny zbiór testowy** pochodzący z innych nagrań niż treningowy, czyli uczciwy egzamin na koniec.
- **Naturalnie trudne zdjęcia.** Część zdjęć jest ciemna, mała albo nieostra już w oryginale, więc temat odporności nie jest wymyślony na siłę.
- **Ramki ROI,** czyli dokładne położenie znaku na każdym zdjęciu. Przydadzą mi się w KM3, kiedy będę sprawdzał, czy model faktycznie patrzy na znak, czy na tło.

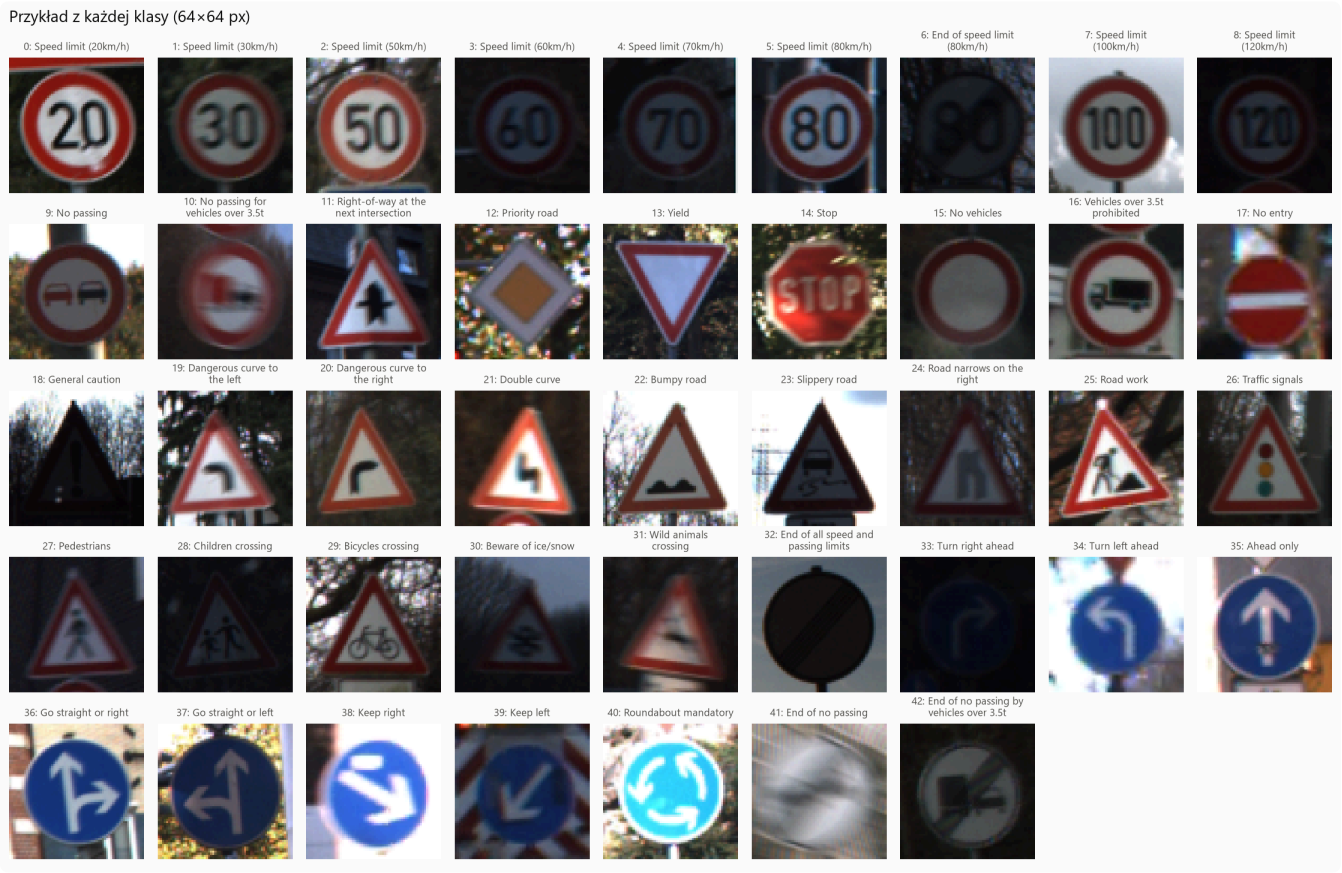
Zbiór testowy od samego początku odłożyłem na bok. W KM1 korzystałem tylko z etykiet i rozmiarów zapisanych w `Test.csv`, żeby porównać rozkłady klas. Samych zdjęć testowych nie otwierałem i użyję ich dopiero w KM3, jeden raz, do oceny finalnego modelu.

3. Analiza eksploracyjna danych

Analizę zrobiłem w notebooku `notebooks/01_eda.ipynb`, a wykresy i policzone statystyki zapisałem w `results/eda/`. Przy każdej obserwacji od razu opisuję, jaką decyzję na jej podstawie podjąłem, bo właśnie po to tę analizę robiłem.

3.1. Przegląd klas

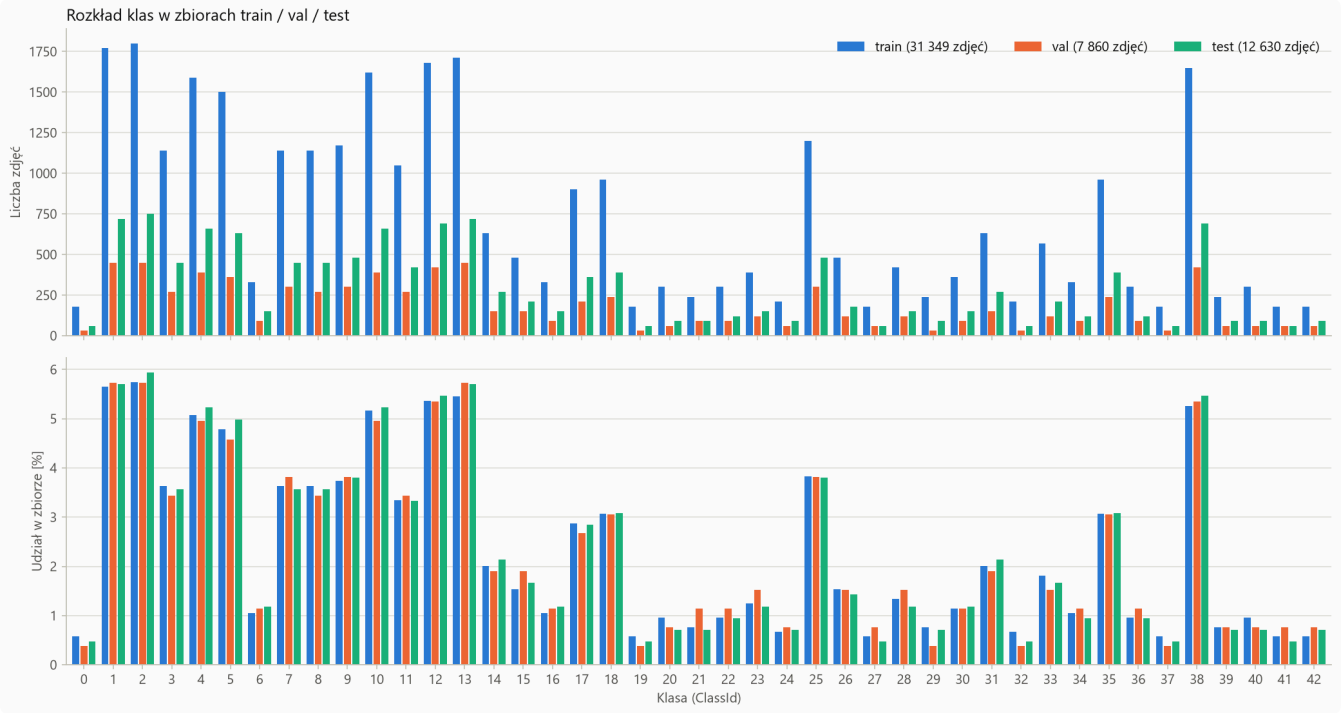
W zbiorze są 43 klasy: ograniczenia prędkości, zakazy, nakazy, ostrzeżenia i znaki informacyjne. Już na pierwszy rzut oka widać, że niektóre są do siebie łudząco podobne. Najbardziej rzucają się w oczy ograniczenia prędkości (20, 30, 50, 60, 70, 80, 100 i 120 km/h), które różnią się wyłącznie cyframi w środku tego samego czerwonego okręgu. Obstawiam, że to właśnie tam model będzie mylił się najczęściej.



Rys. 1. Przykładowe zdjęcie z każdej z 43 klas.

3.2. Liczność klas

Zbiór jest mocno niezbalansowany. Najrzadsza klasa (0, ograniczenie do 20 km/h) ma 210 zdjęć treningowych, a najliczniejsza (2, ograniczenie do 50 km/h) aż 2 250, czyli prawie jedenaście razy więcej. W zbiorze testowym proporcje wyglądają niemal identycznie.

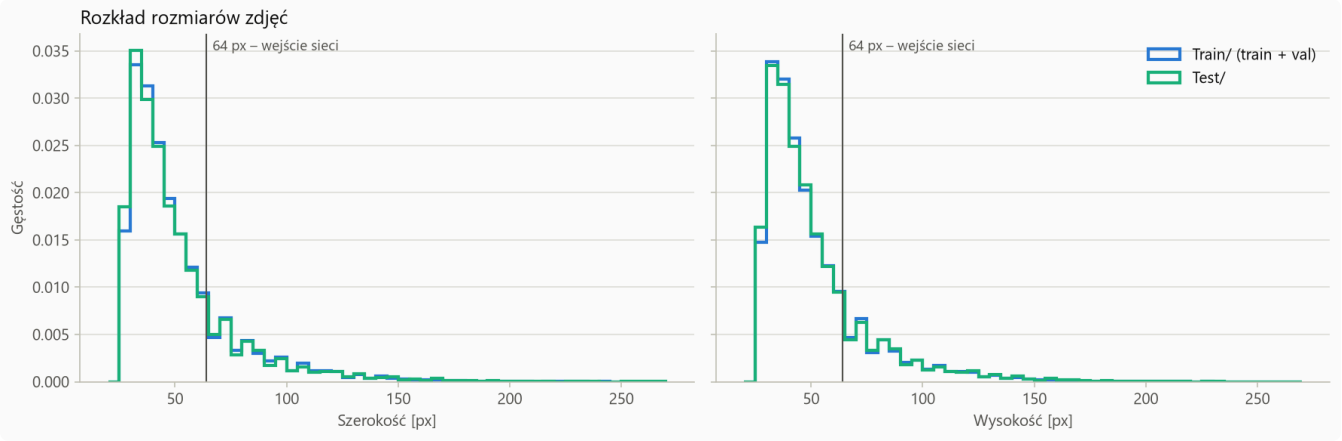


Rys. 2. Liczba zdjęć (góra) oraz udział procentowy (dół) każdej klasy w zbiorze treningowym, walidacyjnym i testowym.

Decyzja: samo accuracy byłoby tu złudne, bo model mógłby mieć świetny wynik ogólny, a jednocześnie regularnie mylić rzadkie znaki. Dlatego jako główną metrykę porównawczą przyjąłem **macro-F1**, w której każda klasa liczy się tak samo, niezależnie od liczby zdjęć.

3.3. Rozmiary zdjęć

Zdjęcia są małe: od 25 do około 250 pikseli szerokości, z medianą 43 px. Prawie 80% z nich ma oba boki krótsze niż 64 px. Proporcje są praktycznie kwadratowe (mediana stosunku szerokości do wysokości wynosi dokładnie 1,0).

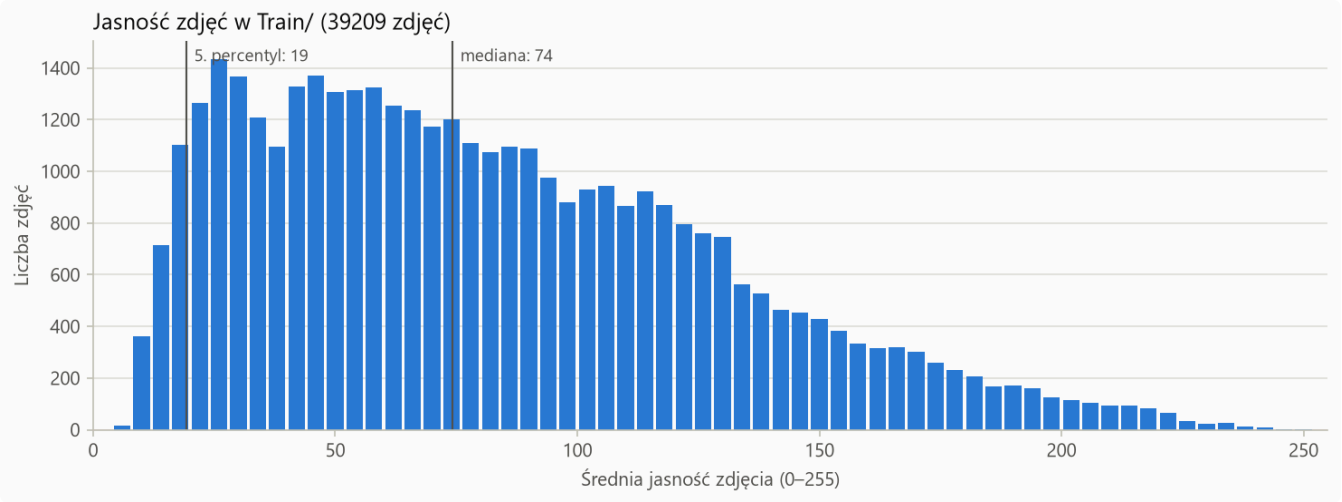


Rys. 3. Rozkład szerokości i wysokości zdjęć.

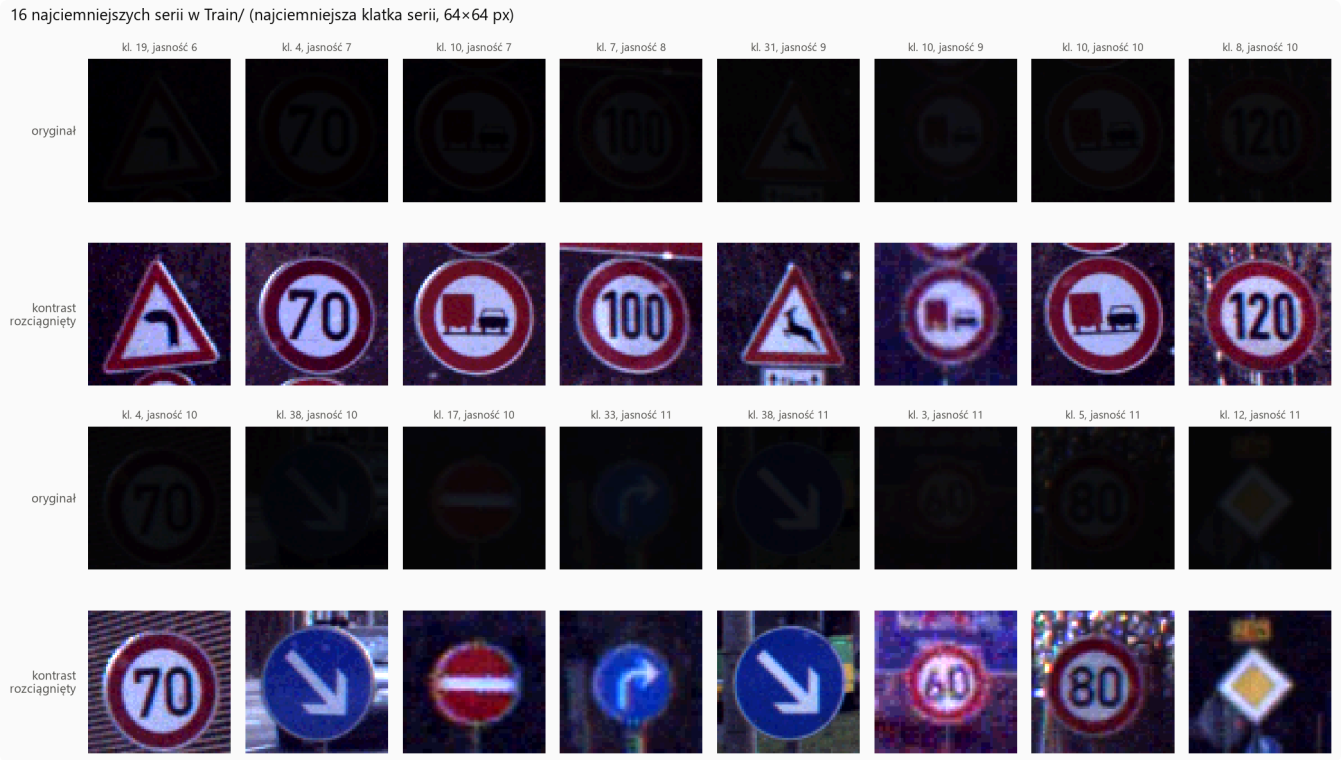
Decyzja: wszystkie zdjęcia skaluję do **64×64 px**. Dla większości danych to lekkie powiększenie, więc nic nie tracę, a trening zostaje szybki. Skoro zdjęcia są kwadratowe, przeskalowanie nie rozciąga znaków.

3.4. Jasność

Ta obserwacja trochę mnie zaskoczyła. Mediana średniej jasności zdjęcia to tylko 74 w skali od 0 do 255, około 14% zdjęć ma średnią jasność poniżej 30, a 30% poniżej 50. Najciemniejsze przykłady są trudne do rozpoznania nawet dla człowieka.



Rys. 4. Rozkład średniej jasności zdjęć treningowych.

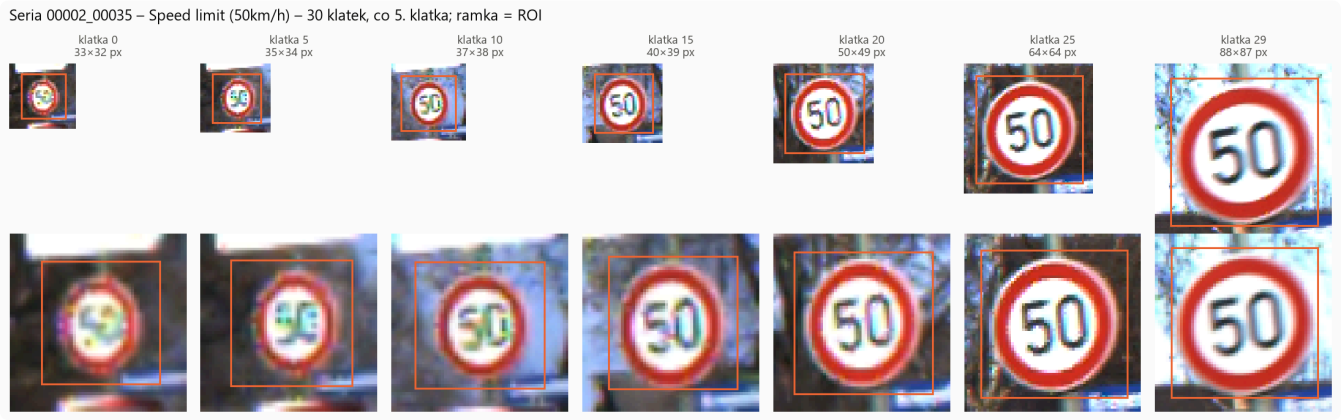


Rys. 5. Przykłady najciemniejszych zdjęć w zbiorze treningowym.

Decyzja: ciemne zdjęcia nie są w GTSRB wyjątkiem, tylko codziennością, i musiałem to uwzględnić w dwóch miejscach. Augmentacja przyciemniająca nie może być zbyt agresywna, bo nałożona na i tak ciemne zdjęcie potrafi całkiem wymazać znak (rozdział 4.3). Z kolei zakłócenie „przyciemnienie", którym będę sprawdzał odporność, musi być na tyle mocne, żeby wychodzić poza to, co model i tak regularnie widzi w danych (rozdział 5.4).

3.5. Serie zdjęć tego samego znaku

To była dla mnie najważniejsza obserwacja całej analizy. Zbiór treningowy nie składa się z 39 tysięcy niezależnych zdjęć, tylko z **1 307 serii** (tracków). Każda seria to około 30 kolejnych klatek nagrania tego samego fizycznego znaku, który rośnie w miarę zbliżania się samochodu. Numer serii da się odczytać z nazwy pliku (`<klasa>_<seria>_<klatka>.png`), a na jedną klasę przypada od 7 do 75 serii.



Rys. 6. Co piąta klatka jednej serii (ograniczenie do 50 km/h). Górny rząd pokazuje oryginalne rozmiary, dolny po przeskalowaniu do 64×64 px. Pomarańczowa ramka to ROI.

Decyzja: sąsiednie klatki są praktycznie identyczne. Gdybym wydzielił walidację losowo po pojedynczych zdjęciach, prawie te same obrazy trafiłyby jednocześnie do treningu i do walidacji, a model „zdawałby egzamin", znając odpowiedzi. Dlatego dane dzielę **po seriach** (rozdział 4.1).

3.6. Ramki ROI

W plikach CSV każde zdjęcie ma zapisane współrzędne ramki wokół znaku. Zwykle ramka jest odsunięta od krawędzi o około 5 px z każdej strony. Sprawdziłem, że po przeskalowaniu do 64×64 px ramki nadal dobrze obejmują znak (widać to na rys. 6). Wykorzystam je w KM3, żeby policzyć, jaka część uwagi modelu z map Grad-CAM przypada na sam znak, a jaka na tło.

4. Przygotowanie danych

Całe przygotowanie danych zamknąłem w jednym module, `src/data.py`. Dzięki temu każdy eksperyment w KM2 będzie korzystał z dokładnie tego samego podziału, przetwarzania i augmentacji, więc różnice w wynikach będą wynikać z modeli, a nie z danych.

4.1. Podział na zbiór treningowy i walidacyjny

Walidację wydzieliłem z oficjalnego zbioru treningowego, zgodnie z wnioskiem z rozdziału 3.5 po seriach: każda seria trafia w całości albo do treningu, albo do walidacji. Użyłem do tego `StratifiedGroupKFold` z biblioteki scikit-learn (5 foldów, grupy to serie, stratyfikacja po klasach, ziarno losowości 42), a jeden fold, czyli około 20% serii, przeznaczyłem na walidację. Stratyfikacja pilnuje, żeby proporcje klas w obu zbiorach były takie same.

Zbiór	Zdjęcia	Serie	Przeznaczenie
Treningowy	31 349	1 045	uczenie modeli
Walidacyjny	7 860	262	wybór wariantu, early stopping, ocena odporności w KM2
Testowy	12 630	brak danych o seriach	jednorazowa ocena finalnego modelu w KM3

Po wykonaniu podziału sprawdziłem, czy wszystko się zgadza:

- trening i walidacja nie mają ani jednej wspólnej serii,
- w walidacji są wszystkie 43 klasy, każda z co najmniej 30 zdjęciami,
- udział żadnej klasy nie różni się między treningiem a walidacją o więcej niż 0,38 punktu procentowego,
- podział jest powtarzalny: listę serii walidacyjnych zapisałem w pliku `splits/val_tracks.json`, który jest w repozytorium i z którego korzystają wszystkie skrypty.

Ograniczenie: klasy 0, 19, 29, 32 i 37 mają w walidacji tylko po jednej serii, bo w całym zbiorze mają ich bardzo mało. Ich wynik walidacyjny opiera się więc na jednym fizycznym znaku i będę go traktował z przymrużeniem oka. Rzetelną ocenę tych klas da dopiero zbiór testowy.

4.2. Przetwarzanie wstępne

Każde zdjęcie przechodzi te same kroki: konwersję do RGB, przeskalowanie do 64×64 px z antyaliasingiem, zamianę na tensor z wartościami od 0 do 1 oraz normalizację średnią i odchyleniem standardowym ze zbioru ImageNet. Ramki ROI skaluję razem ze zdjęciem.

Normalizację ImageNet stosuję dla wszystkich modeli, także dla własnej sieci uczonej od zera, choć ta formalnie jej nie potrzebuje. Chodzi o to, żeby warianty różniły się wyłącznie architekturą i sposobem inicjalizacji wag, a nie tym, jak przygotowano dla nich dane.

Przy okazji trafiłem na problem z wydajnością: na Windowsie samo wczytywanie plików PNG zajmowało około 70 s na epokę. Dodałem więc opcję jednorazowego wczytania i przeskalowania wszystkich zdjęć do pamięci RAM. Wyniki są identyczne co do bitu, a epoka trwa teraz około 13 s. Przy kilkunastu zaplanowanych treningach to robi sporą różnicę.

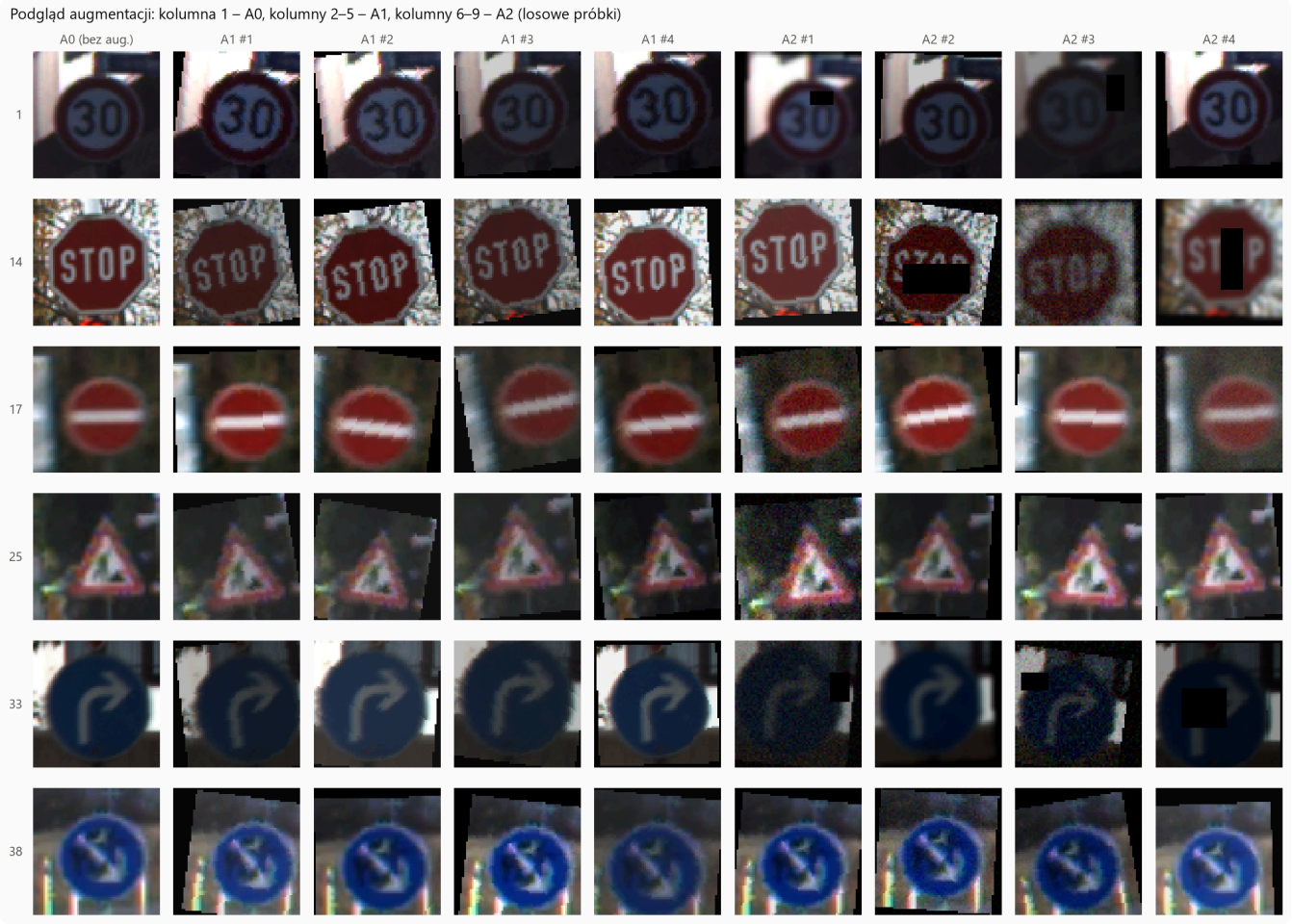
4.3. Augmentacja danych

Przygotowałem trzy poziomy augmentacji, które porównam w KM2:

Poziom	Operacje
A0 brak	tylko przetwarzanie wstępne (ten sam zestaw służy do walidacji i testu)
A1 geometryczna	A0 + losowy obrót ±10°, przesunięcie do 10%, skalowanie 0,9 do 1,1, zmiana jasności i kontrastu ±30%
A2 odpornościowa	A1 + z prawdopodobieństwem 0,3 każda: rozmycie Gaussa (σ 0,1 do 1,5), przyciemnienie (×0,5 do 0,9), szum Gaussa (σ 0,01 do 0,05), wymazanie prostokątnego fragmentu (2% do 15% powierzchni)

Dwie rzeczy wymagają tu wyjaśnienia:

- **Celowo nie używam odbić lustrzanych,** choć to jedna z najpopularniejszych augmentacji. Przy znakach drogowych odbicie zmienia ich znaczenie: odbity nakaz skrętu w prawo staje się nakazem skrętu w lewo, a model dostałby zdjęcie z błędną etykietą.
- **A2 osłabiłem po obejrzeniu danych.** W pierwszej wersji przyciemniała zdjęcia nawet do 30% jasności i dodawała szum o σ do 0,08. Na podglądzie okazało się, że na zdjęciach, które i tak były ciemne, znak potrafił całkowicie zniknąć, więc model uczyłby się na obrazach, których nie da się rozpoznać. Zmniejszyłem zakresy do wartości z tabeli. Zrobiłem to, zanim wytrenowałem jakikolwiek model, wyłącznie na podstawie analizy danych, żeby nie było podejrzeń, że dopasowuję augmentację do wyników.



Rys. 7. Podgląd augmentacji po korekcie. Kolumna 1 to A0, kolumny 2 do 5 to losowe próbki A1, kolumny 6 do 9 to losowe próbki A2.

W żadnej augmentacji nie ma kompresji JPEG i jest to zamierzone. JPEG posłuży mi w KM2 jako zakłócenie, którego model nie zobaczy w treningu ani razu (hipoteza H4).

5. Projekt eksperymentu

Cały plan eksperymentów ustaliłem przed pierwszym treningiem. Każdy eksperyment odpowiada na konkretną hipotezę, a sposób wyboru najlepszego modelu jest zapisany z góry. Dzięki temu w KM2 nie będę na ślepo testował kolejnych modeli, tylko sprawdzał założenia z tego rozdziału.

5.1. Hipotezy

1. **H1 (uczenie transferowe):** modele startujące z wag wytrenowanych na ImageNet osiągną wyższe macro-F1 i szybciej się nauczą niż te same architektury uczone od zera.
2. **H2 (augmentacja):** augmentacja odpornościowa A2 zmniejszy spadek skuteczności na zakłóconych zdjęciach wyraźnie bardziej niż geometryczna A1, a na czystych danych straci najwyżej 1 punkt procentowy macro-F1.

3. **H3 (rozmiar modelu):** MobileNetV3-Small osiągnie wynik zbliżony do ResNet18 (różnica macro-F1 poniżej 1 punktu procentowego), mając przy tym kilka razy mniej parametrów i krótszy czas predykcji.
4. **H4 (uogólnienie odporności):** odporność nabyta dzięki augmentacji przeniesie się przynajmniej częściowo na zakłócenie, którego model nie widział w treningu (kompresja JPEG).

5.2. Baseline

Jako punkt odniesienia przyjąłem **własną, prostą sieć konwolucyjną (SimpleCNN)** uczoną od zera i bez augmentacji. Ma trzy bloki (konwolucja, normalizacja wsadowa, ReLU, max pooling) i klasyfikator z warstwą w pełni połączoną. Bez takiego punktu odniesienia trudno byłoby ocenić, czy wynik bardziej złożonych modeli to faktycznie sukces, czy coś, co da się osiągnąć najprostszym podejściem.

5.3. Warianty rozwiązania

Warianty różnią się w trzech wymiarach:

- **architektura:** SimpleCNN, ResNet18 [3] i MobileNetV3-Small [4],
- **inicjalizacja wag:** od zera albo wagi pretrenowane na ImageNet (dla ResNet18 i MobileNetV3),
- **augmentacja:** A0, A1 lub A2 (rozdział 4.3).

ResNet18 wybrałem jako sprawdzoną sieć średniej wielkości, a MobileNetV3-Small jako lekką sieć projektowaną z myślą o urządzeniach mobilnych, co dobrze pasuje do zastosowania w samochodzie. Wszystkie warianty trenuję w identycznych warunkach: wejście 64×64 px, optymalizator AdamW, najwyżej 20 epok, early stopping na macro-F1 walidacji z cierpliwością 5 epok i ziarno losowości 42.

5.4. Zakłócenia do oceny odporności

Odporność zmierzę na zbiorze walidacyjnym z nałożonymi zakłóceniami, wzorując się na metodologii z badań nad odpornością sieci na typowe zniekształcenia obrazu [5]. Wybrałem pięć zakłóceń, z których każde odpowiada czemuś, co realnie może spotkać kamerę w samochodzie, i każde występuje w trzech poziomach nasilenia. Zakłócenia są deterministyczne (stałe ziarno), więc każdy model dostaje dokładnie te same zepsute obrazy.

Zakłócenie	Odpowiednik w rzeczywistości	Poziom 1	Poziom 2	Poziom 3	Obecne w A2
Rozmycie Gaussa	ruch pojazdu, brak ostrości	σ = 0,8	σ = 2,0	σ = 3,0	tak (σ do 1,5)

Zakłócenie	Odpowiednik w rzeczywistości	Poziom 1	Poziom 2	Poziom 3	Obecne w A2
Szum Gaussa	słabe oświetlenie, tania kamera	σ = 0,03	σ = 0,06	σ = 0,10	tak (σ do 0,05)
Przyciemnienie	zmierzch, noc	×0,6	×0,4	×0,25	tak (do ×0,5)
Zasłonięcie	gałąź, naklejka, śnieg	10% ramki ROI	20% ramki ROI	30% ramki ROI	tak (wymazanie fragmentu)
Kompresja JPEG	słaba jakość obrazu z kamery	jakość 30	jakość 15	jakość 8	**nie**

Dla rozmycia, szumu i przyciemnienia dobrałem poziomy tak, żeby pierwszy mieścił się w zakresie, który model widzi w treningu z A2, a drugi i trzeci wychodziły poza niego. Dzięki temu sprawdzę nie tylko, czy model radzi sobie z tym, co już zna, ale też czy jego odporność sięga dalej. Zasłonięcie umieszczam wewnątrz ramki ROI, bo zakrycie kawałka tła niczego by nie testowało. Przed pierwszym treningiem w KM2 jeszcze raz obejrzę zakłócone obrazy, żeby upewnić się, że parametry mają sens.

5.5. Metryki

Metryka	Co mierzy	Hipoteza
Accuracy i macro-F1 na czystych danych	ogólną skuteczność, macro-F1 z równą wagą dla rzadkich klas	H1, H3
mCA (mean Corruption Accuracy)	średnią accuracy po 5 zakłóceniach i 3 poziomach, czyli odporność	H2, H4
Względny spadek	(accuracy na czystych danych minus mCA) podzielone przez accuracy na czystych danych	H2
Liczba parametrów i czas predykcji na CPU	koszt modelu (mediana ze 100 predykcji pojedynczego zdjęcia)	H3
Macierz pomyłek i F1 dla każdej klasy	które klasy się mylą	analiza błędów

5.6. Plan eksperymentów

Etap	Cel	Warianty	Treningów	Hipoteza
E1	architektura i uczenie transferowe	SimpleCNN; ResNet18 i MobileNetV3-Small, każdy od zera i pretrenowany; wszystkie z A0	5	H1, H3
E2	wpływ augmentacji	SimpleCNN oraz najlepszy wariant ResNet18 i MobileNetV3 z E1, każdy z A1 i A2	6	H2
E3	ocena odporności	wszystkie 11 modeli z E1 i E2 ocenione na zakłóconej walidacji	0 (tylko ewaluacja)	H2, H4
E4	stabilność wyniku	zwycięski wariant z trzema różnymi ziarnami losowości	3	wiarygodność wyniku

Łącznie daje to 14 treningów. Przy zmierzonym czasie epoki (13 do 19 s) cały KM2 powinien zamknąć się w około godzinie samego treningu na laptopowej karcie graficznej.

5.7. Reguła wyboru najlepszego rozwiązania

Regułę wyboru zapisałem przed treningami, żeby nie kusiło mnie dopasowanie jej do wyników:

1. odrzucam warianty, których macro-F1 na czystej walidacji jest niższe od najlepszego o więcej niż 1 punkt procentowy,
2. spośród pozostałych wybieram wariant z najwyższym mCA,
3. jeśli różnica mCA jest mniejsza niż 0,5 punktu procentowego, wygrywa model mniejszy i szybszy.

Zbiór testowy nie bierze udziału w żadnym z tych kroków. Użyję go raz, w KM3, do oceny zwycięskiego modelu i baseline'u.

6. Środowisko i reprodukowalność

Element	Wartość
Język i biblioteki	Python 3.12, PyTorch 2.14 z torchvision, scikit-learn, pandas, matplotlib, pytorch-grad-cam, Gradio
Sprzęt	laptop z kartą NVIDIA RTX 4070 Laptop (8 GB VRAM), CUDA 13.2
Repozytorium	github.com/BPajda/GTSRB_robustness

Zadbałem o to, żeby wyniki dało się odtworzyć:

- wszędzie ustawiam to samo ziarno losowości (42): w Pythonie, NumPy, PyTorch i loaderach danych,
- podział na trening i walidację jest zapisany w repozytorium (`splits/val_tracks.json`), więc nie losuje się od nowa przy każdym uruchomieniu,
- wersje bibliotek są przypięte w `requirements.txt`, a `README.md` opisuje instalację w wersji z CUDA i bez,
- notebook EDA jest zapisany razem z wynikami, więc analizę można przejrzeć bez uruchamiania kodu.

Samego zbioru danych nie umieściłem w repozytorium, bo jest za duży. W `README.md` opisałem, skąd go pobrać i gdzie rozpakować.

```
GTSRB_robustness/
├── data/            # zbiór GTSRB (poza repozytorium)
├── splits/          # zapisany podział train/val
├── notebooks/       # 01_eda.ipynb
├── src/             # data.py oraz moduły przygotowane na KM2 i KM3
├── results/eda/     # wykresy i statystyki z EDA
├── configs/         # konfiguracje eksperymentów (KM2)
├── app/             # aplikacja demonstracyjna (KM3)
└── docs/            # sprawozdania
```

7. Podsumowanie i plan na KM2

W pierwszym kamieniu milowym:

- określiłem problem i pytanie badawcze, czyli odporność klasyfikatora znaków drogowych na zakłócenia obrazu,

- wybrałem zbiór GTSRB, uzasadniłem ten wybór i przeprowadziłem analizę eksploracyjną,
- na podstawie analizy podjąłem konkretne decyzje: macro-F1 jako główna metryka (nierównowaga klas), wejście 64×64 px (małe zdjęcia), podział po seriach (niemal identyczne klatki), osłabienie augmentacji A2 i dobór siły zakłóceń (dużo ciemnych zdjęć) oraz rezygnacja z odbić lustrzanych (zmiana znaczenia znaków),
- przygotowałem dane: powtarzalny podział bez wycieku między zbiorami, przetwarzanie wstępne i trzy poziomy augmentacji,
- zaplanowałem eksperyment: cztery hipotezy, baseline, warianty, pięć zakłóceń w trzech poziomach, metryki i regułę wyboru modelu ustaloną przed treningami.

Na pytanie, co dokładnie chcę sprawdzić i jak udowodnię, że rozwiązanie działa, odpowiadają hipotezy H1 do H4 i metryka mCA. Jeśli model jest odporny, jego skuteczność na zakłóconych zdjęciach będzie niewiele niższa niż na czystych, a porównanie wariantów pokaże, który z nich radzi sobie najlepiej i czym różni się od pozostałych.

W KM2 zaimplementuję modele i zakłócenia, przeprowadzę eksperymenty E1 do E4 zgodnie z rozdziałem 5.6, zweryfikuję hipotezy i wybiorę finalny model. Ten model ocenię następnie na zbiorze testowym i poddam analizie wyjaśnialności w KM3.

Bibliografia

1. J. Stallkamp, M. Schlipsing, J. Salmen, C. Igel, *The German Traffic Sign Recognition Benchmark: A multi-class classification competition*, IJCNN 2011. PDF
2. GTSRB, German Traffic Sign Recognition Benchmark, kopia zbioru w serwisie Kaggle. kaggle.com
3. K. He, X. Zhang, S. Ren, J. Sun, *Deep Residual Learning for Image Recognition*, CVPR 2016. arXiv:1512.03385
4. A. Howard i in., *Searching for MobileNetV3*, ICCV 2019. arXiv:1905.02244
5. D. Hendrycks, T. Dietterich, *Benchmarking Neural Network Robustness to Common Corruptions and Perturbations*, ICLR 2019. arXiv:1903.12261